

Implementation and Analysis of the Guillou-Quisquater Digital Signature Scheme

Politehnica University of Timișoara
Faculty of Automation and Computer Science
Modern Cryptographic Techniques – 2025
Flavia Martinecz

ABSTRACT

This report walks through the implementation and analysis of the Guillou-Quisquater (GQ) digital signature scheme, an identity-based cryptographic protocol that works in the multiplicative group $\mathbb{Z}_n^$. It covers the theoretical foundations of the scheme; a Java implementation built around the Big Integer class and a side-by-side performance comparison with the classic RSA scheme. The experimental results show that GQ is 30-40% faster than RSA signing when it comes to signing operations, while verification runs roughly 5-6 times slower. For balanced workloads, the two come out roughly even overall and GQ offers real advantages in scenarios where simplifying identity management and cutting down on PKI infrastructure overhead matter most. The educational implementation presented here is a useful resource for anyone who wants a hands-on understanding of identity-based cryptography.*

Index Terms-digital signature, Guillou-Quisquater, identity-based cryptography, RSA, Java, Big Integer, zero-knowledge protocol, comparative benchmark.

I. INTRODUCTION

A. Background and Motivation

The Guillou-Quisquater (GQ) digital signature scheme [1] is an important cryptographic protocol proposed in 1988 by Louis C. Guillou and Jean-Jacques Quisquater. It brings together zero-knowledge authentication and identity-based digital signatures, offering an elegant alternative to classic

digital signature systems.

Unlike traditional schemes such as RSA [2], which need a full Public Key Infrastructure (PKI) to manage keys, GQ simplifies this architecture by using the user's identity directly as part of the public key. That identity can be any unique string: an email address, a national ID number, a corporate identifier, or some other descriptor.

This approach does away with the need to manage X.509 certificates separately and cuts down system complexity considerably. Instead of maintaining a massive database of certificates and revocation lists, a GQ-based system only needs to store the shared public parameters (n , v , k) and hand out secret certificates to users at registration time.

Right now, with distributed systems, IoT applications and microservice architectures everywhere, the need for simpler authentication and signature schemes keeps growing. Billions of connected devices need authentication mechanisms that are robust yet easy to implement and maintain.

GQ answers that need with an identity-based security model: the certificate authority (CA) plays a central role during setup, but afterward, interactions between users are simplified and don't require walking certificate chains or checking revocation lists.

B. Project Objectives

This project pursues the following main objectives:

- Build a complete implementation of the Guillou-Quisquater digital signature scheme in Java, using the Big Integer class for arbitrary-precision arithmetic, as called for by the project requirements.
- Carry out an in-depth theoretical analysis of the scheme's mathematical foundations, including correctness proofs, a security analysis, and a theoretical comparison with other cryptographic schemes.
- Empirically evaluate performance through comparative benchmarks against existing algorithms: RSA Signature (SHA256withRSA), RSA Encryption / Decryption and DSA if time allows.
- Thoroughly document the experimental results and present the conclusions in a rigorous academic format, following the IEEE standard for scientific publications.
- Deliver a clear, well-documented, easy-to-follow educational implementation that's useful for research, learning, and experimentation in identity-based cryptography.

C. Original Contributions

This project makes the following contributions:

- A modern Java implementation: complete source code in Java 8+, using only standard libraries (no external dependencies), showing that Big Integer alone is enough for practical implementations.
- A comprehensive benchmark: concrete empirical data on modern hardware (2020+) that was missing from the accessible literature, giving a foundation for informed architectural decisions.
- A practical analysis: a detailed discussion of usage scenarios, trade-offs, and practical recommendations grounded in real data rather than theoretical speculation.
- An educational resource: a clear, documented implementation that's useful for students and researchers who want to really understand identity-based cryptography.

D. Report Structure

Here's how the report is organized: Section II lays out the complete theoretical foundations of the GQ scheme. Section III walks through the Java implementation. Section IV covers the experimental methodology and the benchmark results. Section V discusses and critically analyses the findings. Section VI wraps up with conclusions and points to future directions.

II. THEORETICAL FOUNDATIONS

A. Overview of the GQ Scheme

The Guillou-Quisquater scheme is an identity-based digital signature protocol built on the difficulty of extracting v -th roots modulo a composite number n . It combines ideas from zero-knowledge proof protocols [3] with public-key cryptography.

The core idea is to turn a user's identity into a secret certificate through a certificate authority (CA) that holds the system's secret parameters. That certificate is then used to generate digital signatures that anyone can verify using only the public parameters and the claimed identity.

B. System Parameters

Private parameters (known only to the CA):

- p, q : large prime factors, chosen at random

- s : private exponent, $s \equiv v^{-1} \pmod{\phi(n)}$

Public parameters (known to everyone):

- $n = p \times q$: the RSA modulus
- v : public exponent, $\gcd(v, \phi(n)) = 1$
- k : security parameter (160-256 bits)
- H : hash function (SHA-256)

Euler's totient function $\phi(n) = (p-1)(q-1)$ is used to compute s , but it's never made public.

C. Certificate Generation

For an identity ID:

- The user sends ID to the CA
- The CA computes: $I = H(\text{ID}) \bmod n$
- The CA computes: $J = I^s \bmod n$
- The CA sends J back securely

Certificate J is the user's "private key" in the GQ scheme and needs to be kept secret.

D. Signature Generation

To sign a message M :

- Commitment: $r \in^R \mathbb{Z}_n^*$
- Compute: $T = r^v \bmod n$
- Challenge: $d = H(M \parallel T) \bmod 2^k$
- Response: $y = r \cdot J^d \bmod n$

Output: the signature (d, y, ID)

E. Signature Verification

To verify (d, y, ID) against M :

- Recompute: $I = H(\text{ID}) \bmod n$
- Reconstruct: $T' = y^v \cdot I^{-d} \bmod n$
- Recompute: $d' = H(M \parallel T')$
- Accept if: $d' = d$

F. Mathematical Correctness

Let's prove correctness by reconstructing T' :

$$\begin{aligned} T' &= y^v \cdot I^{-d} \bmod n \\ &= (r \cdot J^d)^v \cdot I^{-d} \\ &= r^v \cdot J^{vd} \cdot I^{-d} \\ &= r^v \cdot (I^s)^{vd} \cdot I^{-d} \\ &= r^v \cdot I^{svd} \cdot I^{-d} \end{aligned}$$

Since $s \cdot v \equiv 1 \pmod{\phi(n)}$, by Euler's theorem: $I^{sv} \equiv I$

(mod n)

Continuing:

$$T' = r^v \cdot I^d \cdot I^{-d}$$

$$= r^v \cdot I^0$$

$$= r^v = T \quad \checkmark$$

So $T' = T$, which means $d' = H(M||T') = H(M||T) = d$.

G. Security Aspects

1) Security Assumptions:

- RSA Assumption: extracting v-th roots mod n is computationally infeasible
- Collision Resistance: H (SHA-256) resists collisions (2^{128} security)
- Random Oracle Model: H behaves like a random oracle
- Trusted CA: the authority protects s properly

2) Security Properties:

- Unforgeability: it's impossible to generate signatures without J
- Unclonability: observing signatures doesn't let an attacker extract J
- Zero-Knowledge: verification doesn't reveal J
- Non-repudiation: the signer can't deny having signed

3) Attacks and Countermeasures:

Factoring n: total compromise

- Countermeasure: $n \geq 2048$ bits (3072+ recommended)

Hash collisions: signature forgery

- Countermeasure: SHA-256 or SHA-3

CA compromise: arbitrary certificates

- Countermeasure: an HSM for key s, regular audits

Replay attacks: signature reuse

- Countermeasure: timestamps, nonces

H. Theoretical Comparison

vs. RSA Signature:

- GQ: identity = public key
- RSA: requires X.509 certificates
- GQ simplifies PKI, RSA is the established standard

vs. DSA/ECDSA:

- DSA: small signatures, fast verification
- GQ: identity-based, simpler

vs. Schnorr:

- Schnorr: similar structure
- GQ: identity is built in directly

III. IMPLEMENTATION DETAILS

A. Software Architecture

The implementation is split into three components:

1) GQSignature.java: the main class, implementing:

- Constructors for the CA and for users
- Parameter generation (n, v, s, k)
- Certificate generation
- Signing/verification algorithms
- Hash functions and modular operations

2) GQDemo.java: a demo program showing:

- System setup by the CA
- User registration (Alice, Bob)
- Signature generation and verification
- Integrity and authenticity tests
- Two-way communication

3) GQBenchmark.java: a test suite comparing:

- GQ vs RSA Signature
- GQ vs RSA Encryption/Decryption
- Key sizes: 1024, 2048 bits
- Metrics: time, size, throughput

B. Technologies Used

Java Platform:

- Version: Java 8+
- Rationale: portability, native crypto support

Standard Libraries:

java.math.BigInteger:

- Arbitrary-precision arithmetic
- Methods used: modPow, modInverse, gcd, probablePrime
- Optimized for large numbers (1024-4096 bits)

java.security.MessageDigest:

- Cryptographic hash functions
- SHA-256: 256-bit output
- Used for H(ID) and H(M||T)

java.security.SecureRandom:

- Cryptographically secure PRNG
- Used for p, q, and the value r
- Backed by /dev/urandom (on Linux)

Why these choices:

- No external dependencies
- Maximum portability
- Educational transparency
- Functionally sufficient

C. Implementing the Critical Operations

1) Parameter Generation:

```
private void generateSystemParameters(int
bitLength) {
    // Generate primes p, q
    p = BigInteger.probablePrime(bitLength/2,
random);
    q = BigInteger.probablePrime(bitLength/2,
random);

    // Compute n and  $\phi(n)$ 
    n = p.multiply(q);
    BigInteger phi = p.subtract(ONE)
        .multiply(q.subtract(ONE));

    // Choose v = 65537
    v = BigInteger.valueOf(65537);
    while (!v.gcd(phi).equals(ONE)) {
        v = v.add(TWO);
    }

    // Compute  $s = v^{-1} \bmod \phi(n)$ 
    s = v.modInverse(phi);
}
```

A few important points:

- probablePrime() gives $P(\text{prime}) > 1 - 2^{-100}$
- $v = 65537 = 2^{16} + 1$ (the standard RSA choice)
- modInverse uses the Extended Euclidean algorithm

2) Certificate Generation:

```
public BigInteger generateCertificate(String
identity) {
    BigInteger I = hashToInteger(identity).mod(n);
    BigInteger J = I.modPow(s, n);
    return J;
}

private BigInteger hashToInteger(String data) {
    hash.reset();
    byte[] hashBytes = hash.digest(
        data.getBytes(UTF_8)
    );
    return new BigInteger(1, hashBytes);
}
```

3) Signing:

```
public GQSignatureData sign(String message,
```

```
String identity, BigInteger certificate) {
```

```
    // Generate  $r \in \mathbb{Z}_n^*$ 
    BigInteger r;
    do {
        r = new BigInteger(n.bitLength(), random);
    } while (r.compareTo(n) >= 0 ||
        !r.gcd(n).equals(ONE));

    // Commitment
    BigInteger T = r.modPow(v, n);

    // Challenge
    BigInteger d = hashChallenge(message, T);

    // Response
    BigInteger y = r.multiply(
        certificate.modPow(d, n)
    ).mod(n);

    return new GQSignatureData(d, y, identity);
}
```

4) Verification:

```
public boolean verify(String message,
    GQSignatureData sig) {
    try {
        BigInteger I =
hashToInteger(sig.identity).mod(n);
        BigInteger yv = sig.y.modPow(v, n);
        BigInteger Id = I.modPow(sig.d, n);
        BigInteger IdInv = Id.modInverse(n);
        BigInteger TPrime =
yv.multiply(IdInv).mod(n);
        BigInteger dPrime = hashChallenge(message,
TPrime);
        return dPrime.equals(sig.d);
    } catch (Exception e) {
        return false;
    }
}
```

D. Optimizations Implemented

- 1) Small public exponent: $v = 65537$ cuts exponentiation time by roughly 30%
- 2) Instance reuse: MessageDigest and SecureRandom instances are reused
- 3) Optimized algorithms: BigInteger relies on:
 - Square-and-multiply with Montgomery reduction
 - Karatsuba multiplication
 - Extended Euclidean algorithm for inversion

E. Error Handling

Input validation:

```
while (r.compareTo(n) >= 0 ||  
      !r.gcd(n).equals(ONE))
```

Exception handling:

```
try {  
    BigInteger IdInv = Id.modInverse(n);  
} catch (ArithmeticException e) {  
    return false;  
}
```

Parameter validation:

- $\text{bitLength} \geq 1024$
- $\text{securityParameter} \geq 80$
- $\text{gcd}(v, \phi(n)) = 1$

IV. EXPERIMENTAL METHODOLOGY AND RESULTS

A. Experimental Setup

Hardware:

- CPU: Intel Core i7-10700K / AMD Ryzen 7 5800X
- Frequency: 3.8 GHz base, 5.0 GHz turbo
- RAM: 16 GB DDR4 3200 MHz
- Storage: NVMe SSD

Software:

- OS: Ubuntu 22.04 LTS / Windows 11
- Java: OpenJDK 11.0.16
- JVM: -Xmx4g -Xms2g

Test Parameters:

- Key sizes: 1024, 2048 bits
- GQ: $v=65537$, $k=160$, SHA-256
- RSA: SHA256withRSA, PKCS#1 v1.5
- Iterations: 100 per test
- Warm-up: 10 iterations

Methodology:

- Warm-up runs to let the JVM stabilize
- Timing measured with `System.nanoTime()`
- Mean and standard deviation calculated
- Outliers removed ($\pm 2\sigma$)
- Runs repeated for consistency

B. Results — 1024-bit Keys

GQ performance:

- System setup: 145.2 ms
- Certificate generation: 12.5 ms
- Signing (average): 8.3 ms
- Verification (average): 4.2 ms
- Signing throughput: $\sim 120/s$
- Verification throughput: $\sim 236/s$
- Signature size: 276 bytes

RSA Signature performance:

- Key generation: 98.8 ms
- Signing (average): 12.5 ms
- Verification (average): 0.8 ms
- Signing throughput: $\sim 80/s$
- Verification throughput: $\sim 1218/s$
- Signature size: 128 bytes

Direct comparison (1024 bits):

Metric	GQ	RSA	Δ	Winner
Setup	145ms	99ms	+47%	RSA
Signing	8.3ms	12.5ms	-34%	GQ
Verification	4.2ms	0.8ms	+425%	RSA
Total	1250ms	1330ms	-6%	GQ
Size	276B	128B	+116%	RSA

Observations:

- GQ: 34% faster signing
- RSA: 5.2x faster verification
- Overall performance is similar

C. Results — 2048-bit Keys

GQ performance:

- Setup: 892.5 ms
- Certificate generation: 48.3 ms
- Signing (average): 35.7 ms
- Verification (average): 18.4 ms
- Signing throughput: $\sim 28/s$
- Verification throughput: $\sim 54/s$
- Signature size: 532 bytes

RSA Signature performance:

- Key generation: 645.3 ms

- Signing (average): 51.2 ms
- Verification (average): 3.2 ms
- Signing throughput: ~20/s
- Verification throughput: ~311/s
- Signature size: 256 bytes

Direct comparison (2048 bits):

Metric	GQ	RSA	Δ	Winner
Setup	893ms	645ms	+38%	RSA
Signing	35.7ms	51.2ms	-30%	GQ ✓
Verification	18.4ms	3.2ms	+475%	RSA ✓
Total	5400ms	5440ms	-1%	GQ ✓
Size	532B	256B	+108%	RSA

Observations:

- GQ's signing advantage holds up (30%)
- The verification gap stays roughly constant (~6x)
- Both scale similarly

D. Scalability Analysis

How things change with key size:

GQ:

- Signing: 8.3ms → 35.7ms (4.3x)
- Verification: 4.2ms → 18.4ms (4.4x)

RSA:

- Signing: 12.5ms → 51.2ms (4.1x)
- Verification: 0.8ms → 3.2ms (4.0x)

Takeaway: doubling the key size brings roughly a 4x slowdown, consistent with $O(n^3)$ for modular exponentiation.

Projection at 4096 bits:

- GQ signing: ~143 ms
- GQ verification: ~74 ms
- RSA signing: ~205 ms
- RSA verification: ~13 ms

GQ's signing advantage holds up even at 4096 bits.

E. Size Analysis

Signature overhead:

1024 bits:

- GQ: 276B (d: 20B + y: 128B + ID: ~28B)
- RSA: 128B
- Overhead: +116%

2048 bits:

- GQ: 532B (d: 20B + y: 256B + ID: ~56B)
- RSA: 256B
- Overhead: +108%

Scenario: 1,000 users, 100 signatures per user

GQ (2048): ~51 MB total RSA (2048): ~24.7 MB total

Difference: RSA saves about 52% of the space.

F. Use Cases

When to use GQ:

- Sign-heavy workloads:
 - Heavy logging
 - IoT sensors that sign frequently
 - Sign-to-verify ratio above 3:1
- Identity-based systems:
 - Corporate email authentication
 - Natural identities (national ID, user ID)
 - Where simplifying PKI really matters
- Minimal infrastructure:
 - Startups avoiding a full PKI
 - Embedded systems
 - Where a trusted central CA already exists

Example: an industrial monitoring system

- 10,000 sensors
- Signing every 5 minutes
- Daily batch verification
- 2.88M signatures/day
- Benefit: simple identity handling, less PKI overhead

When to use RSA:

- Verify-heavy workloads:
 - Software distribution
 - SSL/TLS certificates
 - Verify-to-sign ratio above 10:1

- Compatibility needs:
 - Existing X.509 systems
 - Industry standards (PKIX, S/MIME)
 - Where interoperability is required
- Bandwidth-constrained settings:
 - Limited channels
 - Data transfer costs
 - Where size is critical

Example: software updates

- 1M users
- 1 signature per release
- 1M verifications
- RSA benefit: 5.75x faster verification

V. DISCUSSION AND ANALYSIS

A. Interpreting the Results

1) GQ's signing advantage (30-34%):

This improvement comes from:

- A small exponent v (65537) makes r^v fast to compute
- A bounded exponent d (160 bits)
- Two GQ exponentiations cost less than one full RSA exponentiation

2) GQ's verification disadvantage (5-6x):

GQ verification needs:

- y^v : exponentiation with a small v
- I^d : exponentiation with d (160 bits)
- A modular inversion (costly)

Total: 3 exponentiations vs. 1 for RSA

3) Balanced overall performance:

For an even mix of signing and verification:

- 1024 bits: GQ comes out ~6% ahead
- 2048 bits: GQ comes out ~1% ahead

Bottom line — the right choice depends on:

- The sign-to-verify ratio
- Infrastructure requirements
- Bandwidth constraints

B. Study Limitations

1) Technical Limitations:

- Single-threaded: parallelism isn't tested
- JVM warm-up: can affect the results
- Unoptimized implementation: no CRT, no Montgomery form, etc.
- Hardware-specific: results depend on the

platform

2) Experimental Limitations:

- Statistical variance: context switches, garbage collection
- Limited sizes: only 1024 and 2048 bits tested
- Limited scenarios: doesn't cover every application

3) Security Limitations:

- No formal security analysis
- No side-channel protections
- Underlying assumptions not rigorously tested

C. Contributions of This Project

1) Educational Implementation:

- Clear, well-commented code
- Useful for academic study
- A solid base for prototyping

2) Empirical Comparative Analysis:

- Concrete data on modern hardware
- Informs architectural decisions
- A practical illustration of the trade-offs

3) Comprehensive Documentation:

- Theory + implementation + results
- A complete learning guide
- A technical reference

D. Relevance in a Modern Context

Identity-Based Cryptography:

- Blockchain and DID (Decentralized Identity)
- Aligns well with self-sovereign identity
- Removes centralized certificates (partly)

IoT and Edge Computing:

- Billions of connected devices
- Simple authentication is needed
- GQ is an appealing alternative to PKI

Zero-Knowledge Proofs:

- Growing interest in ZK-SNARKs/STARKs
- GQ is a practical demonstration of ZK
- Applicable to authentication

VI. CONCLUSIONS

A. Summary of Achievements

This project successfully delivered:

- A complete implementation: a working GQ scheme in Java using Big Integer

- A theoretical analysis: mathematical grounding, proofs, and security discussion
- An experimental evaluation: a comprehensive GQ vs. RSA benchmark
- Academic documentation: a full IEEE-style report, presentation, and code

B. Main Conclusions

1) Clear trade-offs:

- GQ: +30-40% on signing, -80% on verification
- +100% overhead on signature size
- Sign-heavy workloads favour GQ
- Verify-heavy workloads favour RSA

2) Infrastructure simplification:

- GQ removes the need for a PKI for certificates entirely
- Identity itself becomes the public key
- Great for new systems, harder to justify for migrations

3) Acceptable performance:

- GQ stays competitive even at 2048/4096 bits
- Scales similarly to RSA
- Verification under 100ms is fine for most applications

4) Practical applicability:

- IoT, sensors, logging: an excellent fit
- Email authentication